

Tools for software development

Lab 2 - Unit Testing and Project Selection



doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk

JUnit
+
Projects



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Where we are in the course

Lab 2 connects unit testing with the semester testing project.



Lab 2 outcomes

By the end of today each team should have evidence, not only ideas.

Unit testing

- Create a Java Maven project
- Add JUnit 5 dependency
- Write and run basic assertions
- Create at least one failing test
- Run tests with coverage

Quality thinking

- Test boundaries
- Test invalid inputs
- Use assertThrows for exceptions
- Explain what coverage does not prove
- Commit cleanly to Git

Project choice

- Select a stable web/API app
- Identify 5 testable features
- Decide what is manual vs automated
- Check API availability
- Document risks



Testing levels: from code to acceptance

Today we start at the lowest level, but the semester project will cover several levels.

Acceptance testing

Does it satisfy the user?

System testing

Does the complete system work?

Integration testing

Do components work together?

Unit testing

Does a small unit behave correctly?

Lab 2 focus: JUnit unit tests
Project focus later: manual, UI and API tests



Tools for today

Keep the setup simple. The goal is to write tests, not fight configuration.

Development

- IntelliJ IDEA
- Java JDK
- Maven or Gradle
- Git repository

Testing

- JUnit 5
- Assertions
- assertThrows
- Coverage runner

Project docs

- README.md
- docs/project-selection.md
- reports/lab2-unit-testing.md
- Commit history

```
git status
java -version
mvn -version
```



Creating a new project

- Creating a project:
best via the Maven / Gradle
new maven project
empty source code
- source: https://github.com/jkostolny/ESIEA_Summer_School/
- For a new project, it is necessary to assign dependencies:
dependencies on jUnit
<https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-api>



JUnit 5 dependency

Add JUnit to the project before creating tests.

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Explain

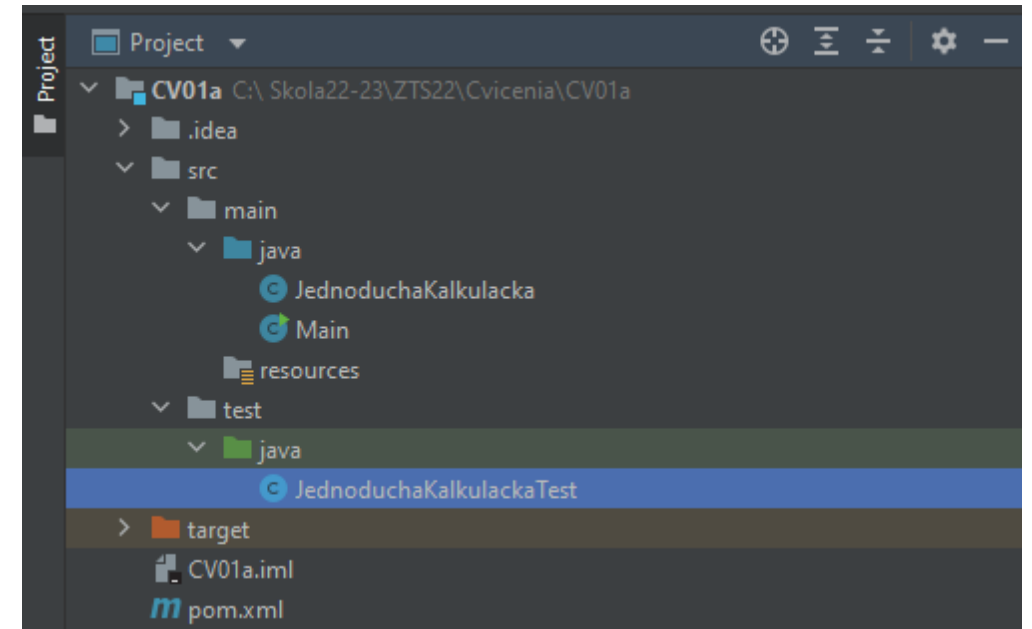
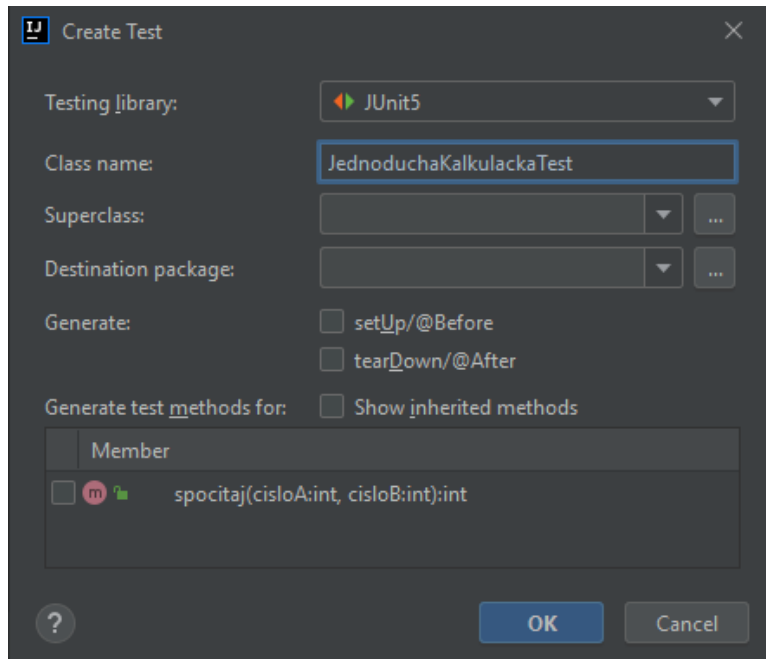
- Dependencies tell Maven which libraries are needed
- JUnit lives in the test scope
- Test code should be under **src/test/java**
- Production code should be under **src/main/java**

Tip: in IntelliJ, use Ctrl + Shift + T on a class to create a test class.



Test creation

- creating a test for a given class
 - keyboard shortcut "**Ctrl + Shift + T**"
- a test will be created:



Exercise: Rating / Grade Calculator

Today we start at the lowest level, but the semester project will cover several levels.

Points	Grade	Boundary cases
93-100	A	92 / 93 <-> 100
85-92	B	84 / 85 <-> 92
77-84	C	76 / 77 <-> 84
69-76	D	68 / 69 <-> 76
61-68	E	60 / 61 <-> 68
0-60	FX	0 <-> 60
Invalid	Exception	-1 / 101



Production code: Rating.java

Implement the logic first, then test it.

```
public class Rating {  
  
    public String determineResultGrade(int points) {  
        if (points < 0 || points > 100) {  
            throw new IllegalArgumentException(  
                "Points must be between 0 and 100");  
        }  
        if (points >= 93) return "A";  
        if (points >= 85) return "B";  
        if (points >= 77) return "C";  
        if (points >= 69) return "D";  
        if (points >= 61) return "E";  
        return "FX";  
    }  
}
```



Test creation

- try tests
 assertion
- Examples:
<https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>
 assert equals
 assert notEquals
 assert true
<https://www.jrebel.com/blog/junit-cheat-sheet>
- Try the test with subtraction and multiplication
- What is the result of the tests after changing to multiplication?
 Are further tests necessary?



First JUnit tests

Start with readable test names and clear expected results.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class RatingTest {

    @Test
    void shouldReturnAFor93Points() {
        Rating rating = new Rating();
        String result = rating.determineResultGrade(93);
        assertEquals("A", result);
    }

    @Test
    void shouldReturnFXFor60Points() {
        Rating rating = new Rating();
        assertEquals("FX", rating.determineResultGrade(60));
    }
}
```



Demonstration of a failed test

```
1 import org.junit.jupiter.api.Test;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class JednoduchaKalkulackaTest {
6
7     @Test
8     void dvaPlusDvaJeStyri() {
9         // JednoduchaKalkulacka kalkulacka = new JednoduchaKalkulacka();
10        var kalkulacka = new JednoduchaKalkulacka();
11        assertEquals("expected: 4, kalkulacka.spocitaj( cisloA: 2, cisloB: 2));", kalkulacka.spocitaj(2, 2));
12        // assertEquals();
13        // assertTrue(kalkulacka.spocitaj(2, 2) == 4);
14    }
15
16     @Test
17     void triPlusSedemJeDesat() {
18        // JednoduchaKalkulacka kalkulacka = new JednoduchaKalkulacka();
19        var kalkulacka = new JednoduchaKalkulacka();
20        assertEquals("expected: 10, kalkulacka.spocitaj( cisloA: 3, cisloB: 7));", kalkulacka.spocitaj(3, 7));
21    }
22 }
```

Run: JednoduchaKalkulackaTest

Tests failed: 1, passed: 1 of 2 tests - 21 ms

Test Results

- JednoduchaKalkulackaTest
- dvaPlusDvaJeStyri() 17 ms
- triPlusSedemJeDesat() 4 ms

org.opentest4j.AssertionFailedError:
Expected :10
Actual :21
<Click to see difference>

<5 internal lines>

- at JednoduchaKalkulackaTest.triPlusSedemJeDesat(JednoduchaKalkulackaTest.java:20) <31 internal lines>
- at java.base/java.util.ArrayList.forEach(ArrayList.java:1511) <9 internal lines>
- at java.base/java.util.ArrayList.forEach(ArrayList.java:1511) <27 internal lines>

Boundary tests

Most defects hide near borders between classes.

```
@Test
void shouldReturnBFor85Points() {
    Rating rating = new Rating();
    assertEquals("B", rating.determineResultGrade(85));
}

@Test
void shouldReturnEFor61Points() {
    Rating rating = new Rating();
    assertEquals("E", rating.determineResultGrade(61));
}

@Test
void shouldReturnFXFor60Points() {
    Rating rating = new Rating();
    assertEquals("FX", rating.determineResultGrade(60));
}
```

Questions

- Which inputs are the most important?
- What happens exactly at 61?
- What happens exactly at 60?
- What happens one point below a grade border?



Testing Ratings

- create an assessment test
- run the test on F for 59 points
- Test launch options:
 - run test
 - run test with coverage

Final rating:

93 and more points - A–excellent

<85, 93) points - B–very good

<77, 85) points - C–good

<69, 77) points - D–satisfactory

<61, 69) points - E–sufficient

less than 61 points - FX–failed

93% classes, 80% lines covered in all classes in scope

Element	Class, %	Method, %	Line, %
com			
images			
java			
javax			
jdk			
META-INF			
netscape			
org			
sun			
toolbarButtonGraphics			
Hodnotenie	100% (1/1)	100% (1/1)	33% (4/12)
JednoduchaKalkulacka	0% (0/1)	0% (0/1)	0% (0/1)
Main	0% (0/1)	0% (0/1)	0% (0/1)

tabase
m
Maven

Invalid input tests

Good tests cover what should not be allowed.

```
@Test
void shouldThrowExceptionForNegativePoints() {
    Rating rating = new Rating();

    assertThrows(IllegalArgumentException.class, () -> {
        rating.determineResultGrade(-1);
    });
}

@Test
void shouldThrowExceptionForMoreThan100Points() {
    Rating rating = new Rating();

    assertThrows(IllegalArgumentException.class, () -> {
        rating.determineResultGrade(101);
    });
}
```



Controlled failure

A failed test is useful when it points to a real defect.

Step 1

- Change one boundary in the code
- Example: points ≥ 85 to points > 85
- Run all tests again

Step 2

- Read the failure message
- Identify expected vs actual result
- Do not guess: inspect the changed line

Step 3

- Fix the code
- Run tests again
- Commit the fix with a meaningful message

```
git add .  
git commit -m "Add rating boundary tests"
```



Additional code testing options

- Link to the calculator:
 - <https://github.com/pH-7/Simple-Java-Calculator.git>
- More information about Unit Testing
 - <https://www.twilio.com/blog/java-junit-effective-unit-tests>

Run tests with coverage

Coverage shows what was executed, not whether tests are good.

What coverage helps with

- Find untested branches
- See whether exception paths run
- Discuss missing cases
- Support refactoring decisions

What it does not prove

- Correct requirements
- Good test data
- Usability
- Security
- Business value

Student output

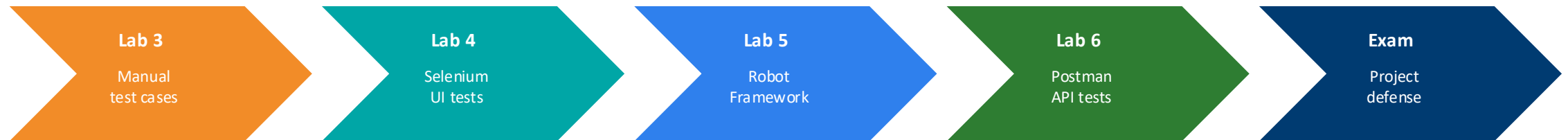
- Screenshot or report
- Short interpretation
- One missing test idea
- Commit to reports folder

Required file: reports/lab2-unit-testing.md



Second part of Lab 2: project selection

The project is the backbone of Labs 3-6 and the final exam.



A weak project choice creates problems later; choose something stable and testable.



Criteria for a good semester project

A good project makes all later labs possible.

Criterion	Why it matters
Accessible without complex setup	Teacher and teammates must be able to reproduce tests
At least 5 testable features	Enough material for manual and automated cases
Forms, search, filters, cart or CRUD	Provides positive and negative scenarios
Stable public demo or own controlled app	Reduces random failures before defense
API available or paired with an API sandbox	Makes Postman Lab 6 possible
No real payments or sensitive personal data	Avoids ethical, legal and practical problems



Recommended project setup

Use one Web UI app plus one API sandbox.

Web UI application

Used for manual test cases
Used for Selenium
Used for Robot Framework
Examples: Sauce Demo, Demo Web Shop, OpenCart Demo



API sandbox

Used for Postman
Used for status codes and JSON assertions
Examples: JSONPlaceholder, ReqRes, Restful Booker, DummyJSON

This combination is robust even when the selected Web UI does not expose a public API.



Project examples: Web UI applications

Best for manual tests, Selenium and Robot Framework.

Project	What to test	Difficulty
Sauce Demo	Login, product list, sorting, cart, checkout validation	Easy
Demo Web Shop	Registration, login, search, product detail, cart, checkout	Medium
OpenCart Demo	Categories, search, product detail, cart, checkout, admin demo	Medium+
Own student web app	CRUD, roles, forms, validation, search, permissions	Depends

If the web app has no public API, pair it with JSONPlaceholder, ReqRes or Restful Booker for Lab 6.



Required document: docs/project-selection.md

This file freezes the project choice for the rest of the course.

```
# Project Selection

## Project name

## Project URL

## Application type
Web / API / both

## Main functionality

## Features selected for testing
1.
2.
3.
4.
5.

## Manual testing possibilities

## Automation possibilities
Selenium / Robot Framework / Postman

## Risks
```



Lab 2 assignment

Two parts: unit testing and project selection.

Part A - Unit testing

- Create Rating.java
- Create RatingTest.java
- Write at least 6 tests
- Include boundary tests
- Include invalid input tests
- Run tests with coverage

Part B - Project selection

- Choose Web UI/API project
- Identify 5 testable features
- Describe manual tests
- Describe automation options
- Document risks
- Push everything to Git

Deadline: end of Lab 2 / end of day 5.7.2026.



Repository output after Lab 2

Everything must be visible in Git.

```
tsd-2026-team-name/  
├── README.md  
├── docs/  
│   ├── team.md  
│   ├── project-charter.md  
│   └── project-selection.md  
├── src/  
│   ├── main/java/Rating.java  
│   └── test/java/RatingTest.java  
└── reports/  
    └── lab2-unit-testing.md
```

Quality checklist

- Tests run successfully
- At least one failing test was demonstrated
- Coverage output is saved
- Project choice is documented
- Git history has meaningful commits



Evaluation for Lab 2

10-point checkpoint for the semester project

Maven/Gradle project and JUnit setup

2 point

At least 5-6 meaningful unit tests

3 point

Boundary and exception testing

2 point

Coverage screenshot/report + short interpretation into documentation

2 point

Documentation committed to repository

1 point

Total: 10 points



Homework before Lab 3

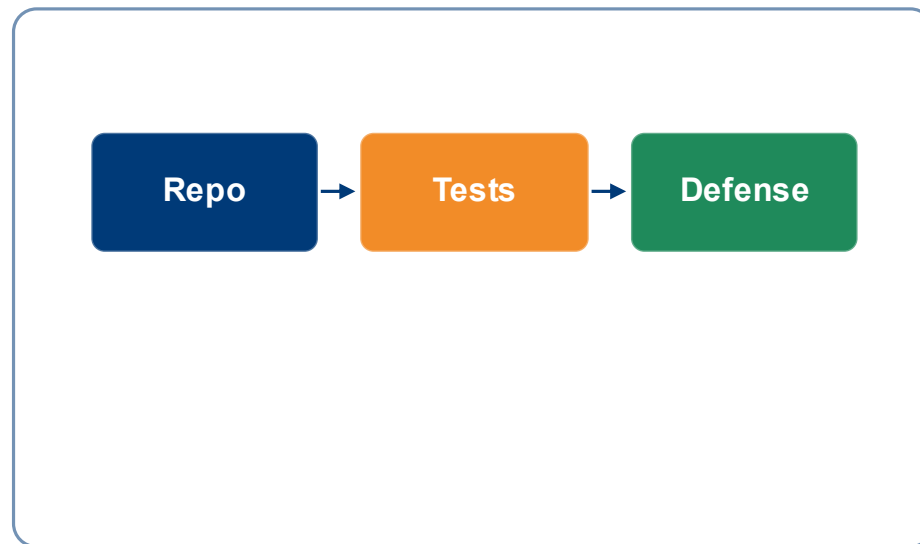
Before you leave, your team should have:

- Rating.java and RatingTest.java
- passing tests + one demonstrated failure
- coverage evidence
- selected Web/API project
- docs/project-selection.md committed to Git

Next lab

Lab 3 – test design techniques and manual test cases





Thank you for your attention.

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk